

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Corso di preparazione per la selezione territoriale delle Olimpiadi di Informatica

Lezione 8. Visite su alberi e grafi

Alessio Martino

28 aprile 2022

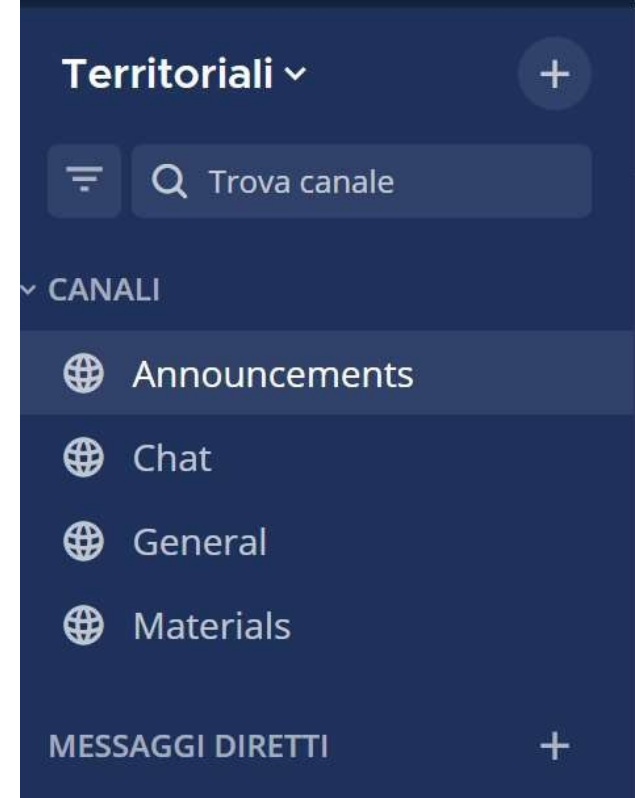
LUISS



Reminder: LUISSTrain

<https://coding.luiss.ml/train/territoriali/>

- 1. *Announcements*** (e.g., link video lezioni)
 - 2. *Chat*** (fai qui le tue domande durante le lezioni)
 - 3. *General*** (discussioni interesse generale)
 - 4. *Materials*** (materiale didattico, slide prima delle lezioni, etc...)
- Controlla i canali: spesso troverai qui le risposte alle tue domande!
 - Non inquinare i canali! Contribuisci a mantenerli puliti e non postare contenuti impropri. Grazie!



Lezioni

Puoi accedere alle lezioni tramite YouTube o Webex:

 **YouTube** (preferibile se hai problemi di connessione)

 **Cisco webex** (limitata a 1000 utenti; però puoi diventare ***Chad (Stacy) per un giorno*** condividendo la tua soluzione ai problemi che consideriamo di volta in volta!)



Programma di oggi

1. Comunicazioni sul Contest Finale (30 aprile)
2. Soluzione di alcuni esercizi della scorsa settimana
3. Alberi e Grafi: Recap
4. Breadth First Search
 - BFS su alberi
 - BFS su grafi
5. Depth First Search
 - DFS su alberi
 - DFS su grafi
6. Algoritmo di Dijkstra
7. Esercizi per casa

Informazioni sul Contest Finale

- Il 30 Aprile si terrà il Contest Intermedio
- Il contest sarà aperto tutto il giorno (dalle 00:00 alle 23:59 del 30 Aprile)
- Potete accedere quando preferite. Una volta effettuato il login, avrete 5 ore di tempo per risolvere i problemi
- Il contest si terrà su <https://terry.luiss.ml>
- Per accedere al Contest sarà necessario un token, che potete richiedere scrivendo al bot https://coding.luiss.ml/train/territoriali/messages/@token_bot#
- Per prendere familiarità con la piattaforma di gara, potete provare a risolvere i problemi delle passate edizioni delle Territoriali su <https://territoriali.olinfo.it>
- Anche in questo caso vi sarà richiesto un token, che corrisponde al vostro username su <https://training.olinfo.it>

Soluzioni di alcuni degli esercizi della scorsa settimana



Disuguaglianze

Scrivete in chat per esporre la vostra soluzione (graph-based):

<https://training.olinfo.it/#/task/disuguaglianze/statement>

Sunnydale

Scrivete in chat per esporre la vostra soluzione:

<https://training.olinfo.it/#/task/sunny/statement>

Spitting the bill (1/5)

Scrivete in chat per esporre la vostra soluzione:

https://training.olinfo.it/#/task/ois_money/statement

Water park (2/5)

Scrivete in chat per esporre la vostra soluzione:

https://training.olinfo.it/#/task/ois_waterslide/statement

Appetito aracnide (3/5)

Scrivete in chat per esporre la vostra soluzione:

<https://training.olinfo.it/#/task/tecla/statement>

Piano di studi (4/5)

Scrivete in chat per esporre la vostra soluzione:

https://training.olinfo.it/#/task/luiss_piano/statement

Incendia lo trabucco (5/5)

Scrivete in chat per esporre la vostra soluzione:

https://training.olinfo.it/#/task/po_trabucco/statement

Alberi e Grafi (Recap)

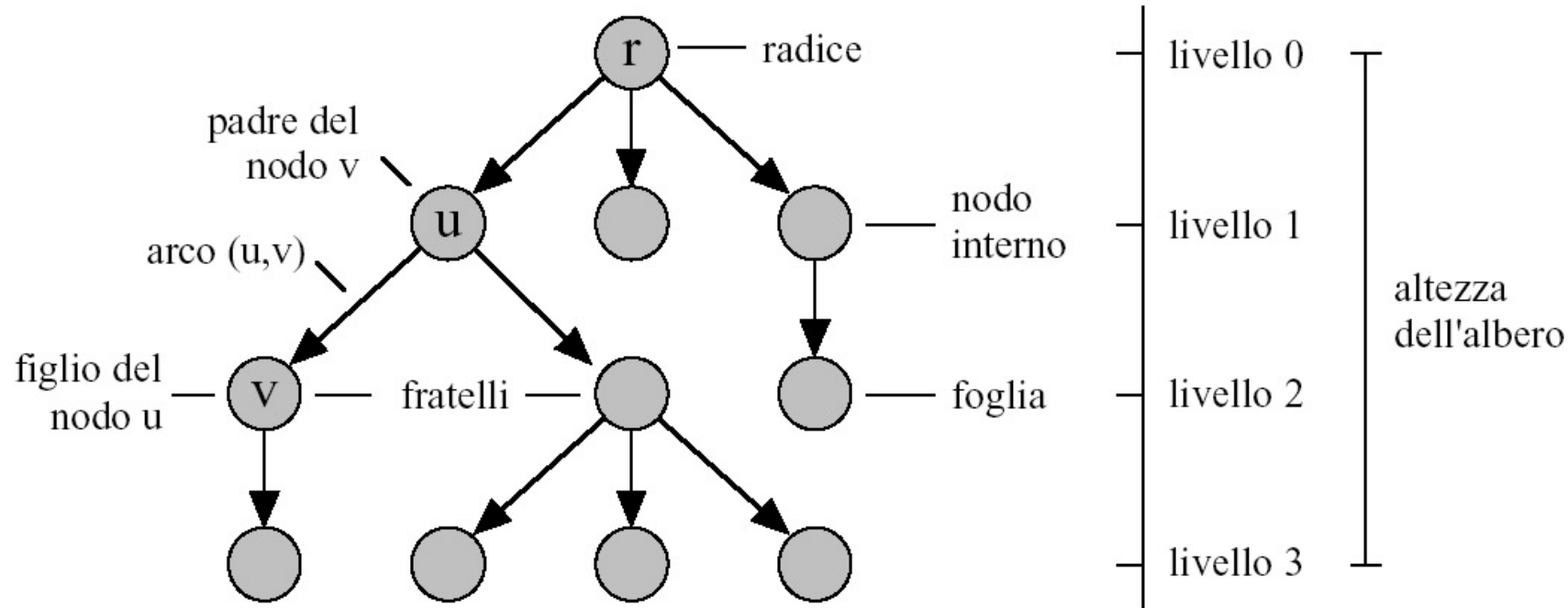
Trees in real life



Trees in computer science



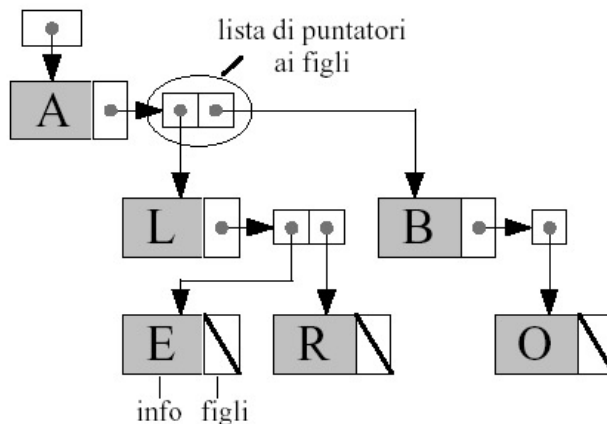
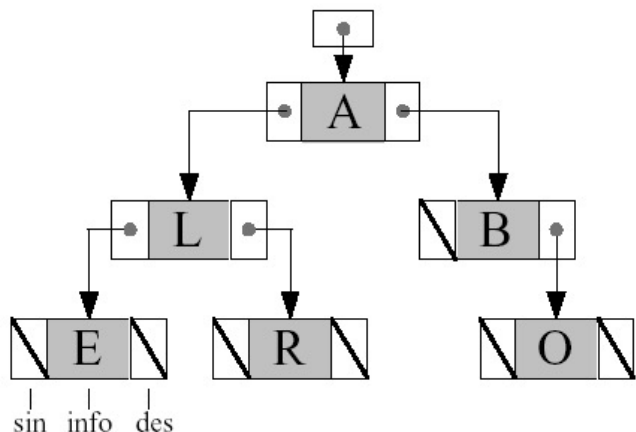
Alberi radicati: grafi aciclici con radice



- organizzazione gerarchica dei dati
 - dati contenuti nei nodi
 - relazioni gerarchiche definite dagli archi che li collegano

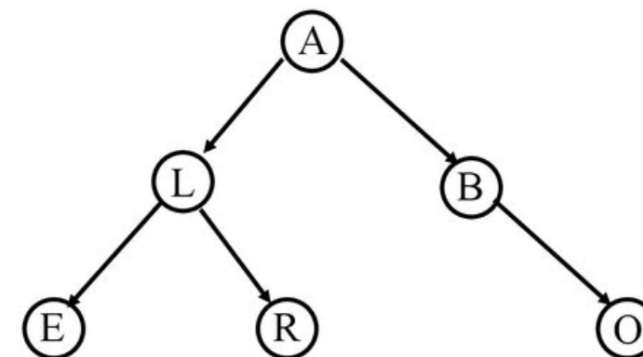
Rappresentazioni di alberi

Rappresentazione con puntatori ai figli (nodi con numero limitato di figli)



Rappresentazione con liste di puntatori ai figli (nodi con numero arbitrario di figli)

Vettore dei padri



(L,3)	(B,3)	(A,null)	(O,2)	(E,1)	(R,1)
1	2	3	4	5	6

Visite di Alberi

- Gli algoritmi di visita di alberi (o grafi) consentono l'**accesso sistematico** ai nodi e agli archi di un albero (o grafo)
- Gli algoritmi di visita si differenziano sostanzialmente in base al particolare **ordine di accesso ai nodi**

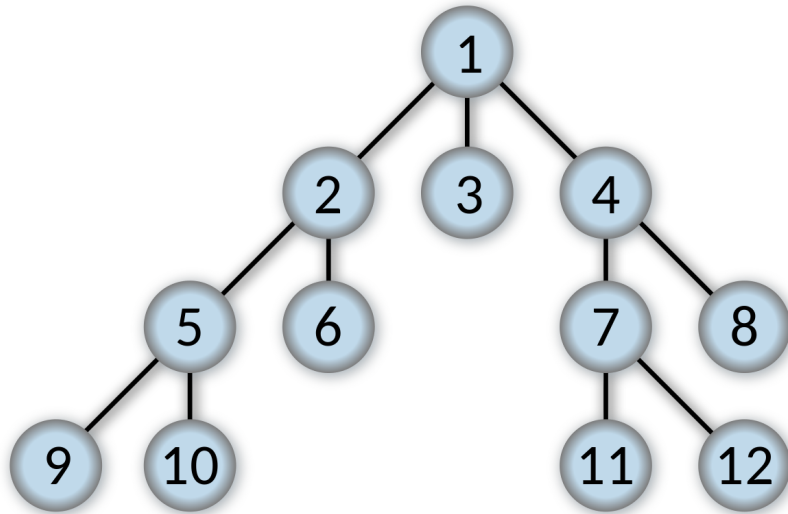


Figura 1: Ricerca in ampiezza (Breadth First Search)

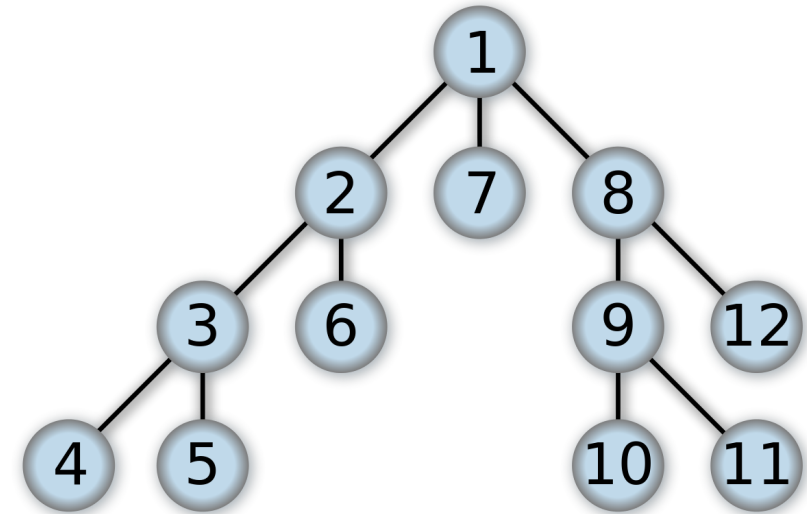


Figura 2: Ricerca in profondità (Depth First Search)

Grafi vs. Alberi

Gli alberi sono una categoria particolare di grafi

- Entrambi sono composti da **nodi** e **archi**
- In entrambi, ogni nodo può avere un qualunque numero di connessioni
 - Se l'albero è binario, c'è un vincolo (max 2 figli per ogni nodo)
- Nei grafi, **esistono cicli**
 - Negli alberi non possono esistere cicli (non si torna mai dai figli al padre)
- Negli alberi, esiste un **nodo** “radice” **univoco**
 - Nei grafi ogni nodo può essere una “radice”

Grafi vs. Alberi

- I grafi, come gli alberi, **possono essere visitati**
 - Visitare un grafo ci permette di esaminare i suoi nodi in **modo sistematico**
- Come per gli alberi, anche per i grafi esistono i due algoritmi di ricerca
 - Breadth First Search (in **ampiezza**)
 - Depth First Search (in **profondità**)
- Diversamente dagli alberi, un nodo può essere incontrato più volte durante la traversata
 - Serve un **sistema di marcatura** per evitare di visitarlo più volte
- Appena un nodo viene visitato (per la prima volta), viene marcato come “visitato”
 - Ad esempio, tramite un bit di marcatura
- La visita genera un **albero di copertura** del grafo

Breadth First Search



Breadth First Search

Descrizione ad alto livello:

1. si parte dalla radice r
2. si visitano tutti i figli di r
3. a turno, ciascun figlio diventa “radice”
4. ripetere da step #2
fino a quando non si raggiungono le foglie

... o in altre parole:

un nodo nel livello i può essere visitato se e soltanto se tutti i nodi al livello $i-1$ sono stati visitati

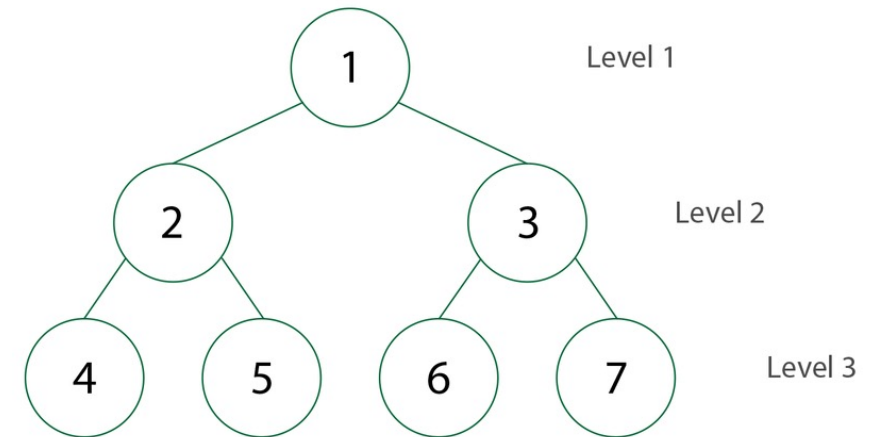


Figura 3: Ricerca in ampiezza (Breadth First Search)

Breadth First Search

Versione iterativa (per alberi binari):

```
function iterative_BFS(graph G, node r):  
    C = empty queue  
    visited = empty list  
    enqueue r in C  
    while C is not empty:  
        dequeue n from C  
        if n is not NULL:  
            add n to visited  
            enqueue left child of n in C  
            enqueue right child of n in C
```

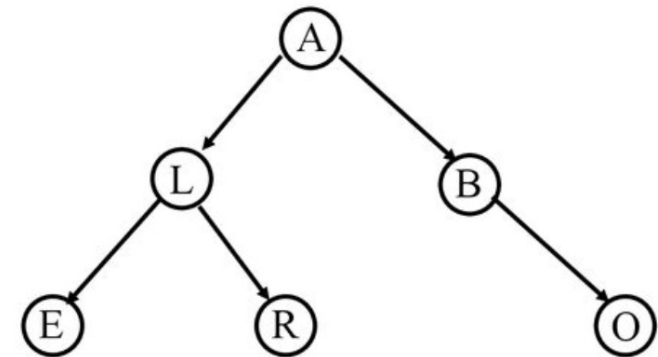


Figura 4: Esempio BFS

Breadth First Search

Inizializzazione

queue = [A]

visited = []

Iterazione #1

queue = [L, B]

visited = [A]

Iterazione #2

queue = [B, E, R]

visited = [A, L]

Iterazione #3

queue = [E, R, O]

visited = [A, L, B]

Iterazione #4

queue = [R, O]

visited = [A, L, B, E]

Iterazione #5

queue = [O]

visited = [A, L, B, E, R]

Iterazione #6

queue = []

visited = [A, L, B, E, R, O]

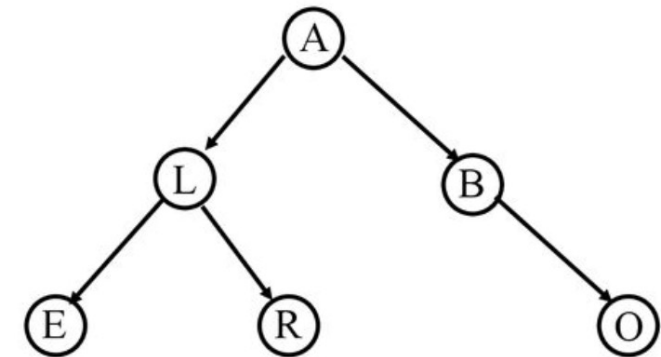


Figura 4: Esempio BFS

Breadth First Search

Versione iterativa (generalizzata):

```
function iterative_BFS(graph G, node n):  
    C = empty queue  
    visited = empty list  
    enqueue n in C  
    add n to visited  
    while C is not empty:  
        dequeue n from C  
        for each node v adjacent to n:  
            if v not in visited:  
                add v to visited  
                enqueue v in C
```

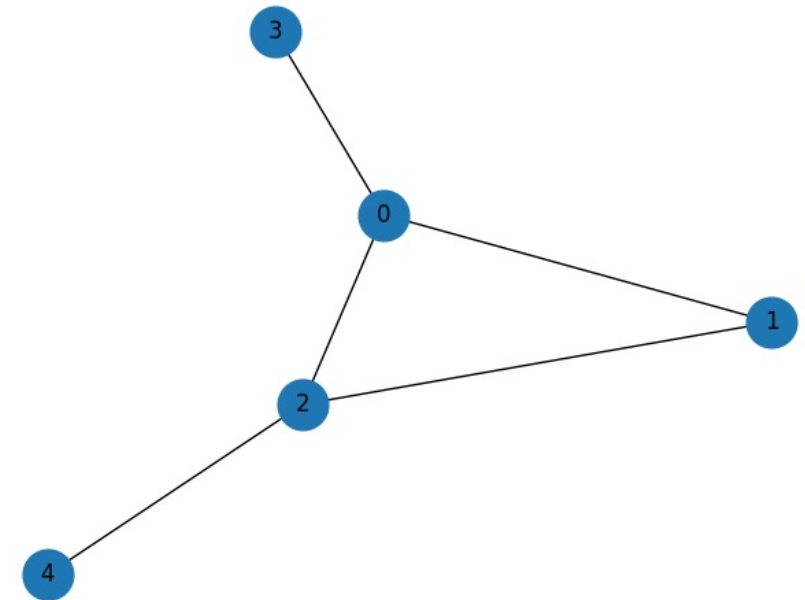


Figura 5: Esempio BFS

Breadth First Search

Inizializzazione

queue = [0]

visited = [0]

Iterazione #1

queue = [1, 2, 3]

visited = [0]

Iterazione #2

queue = [2, 3]

visited = [0, 1]

Iterazione #3

queue = [3, 4]

visited = [0, 1, 2]

Iterazione #4

queue = [4]

visited = [0, 1, 2, 3]

Iterazione #5

queue = []

visited = [0, 1, 2, 3, 4]

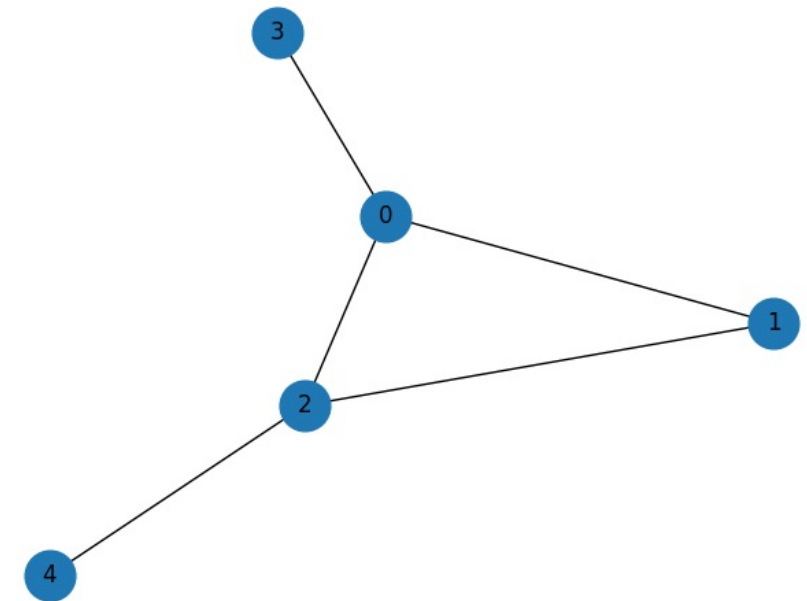


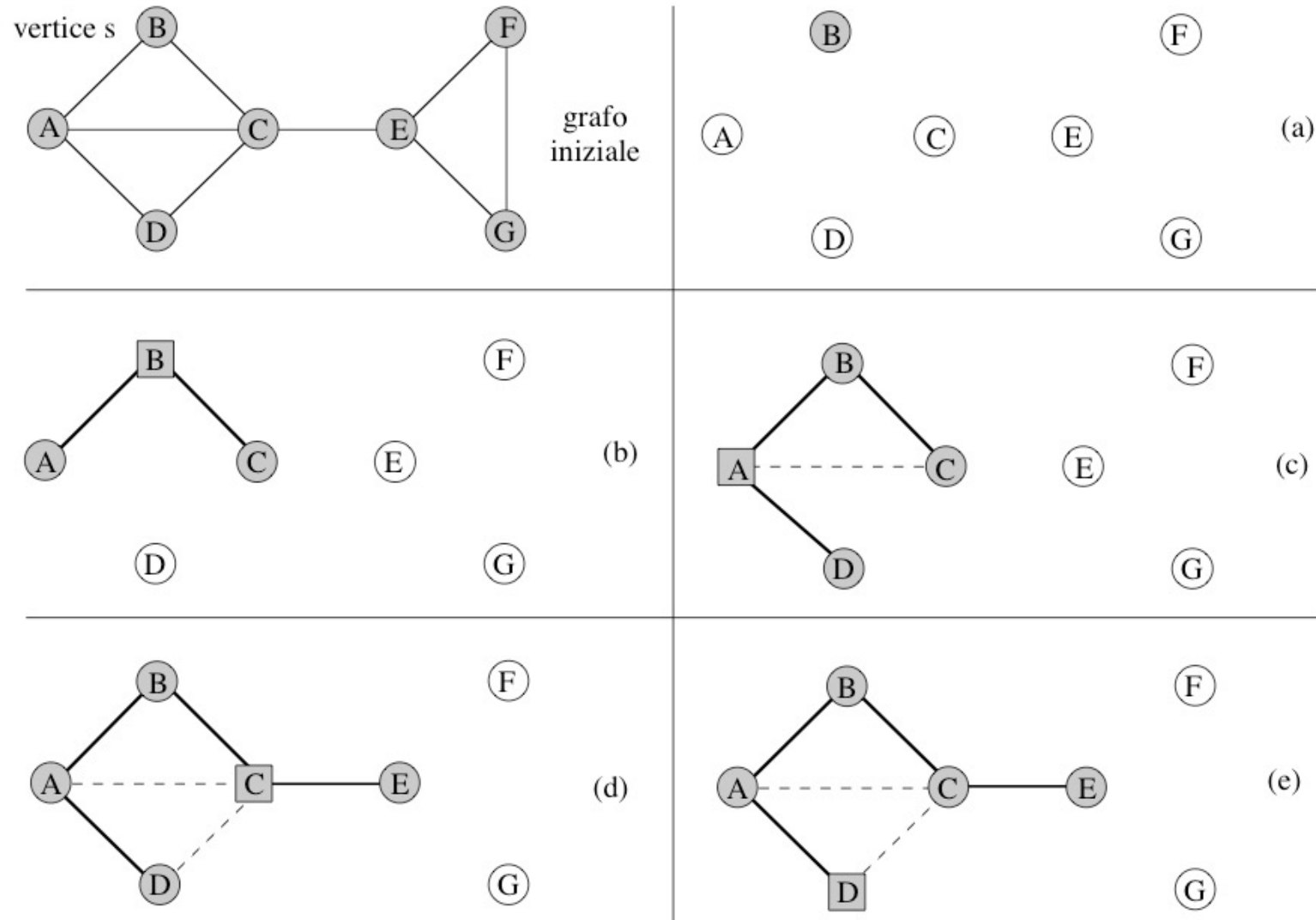
Figura 5: Esempio BFS

Breadth First Search

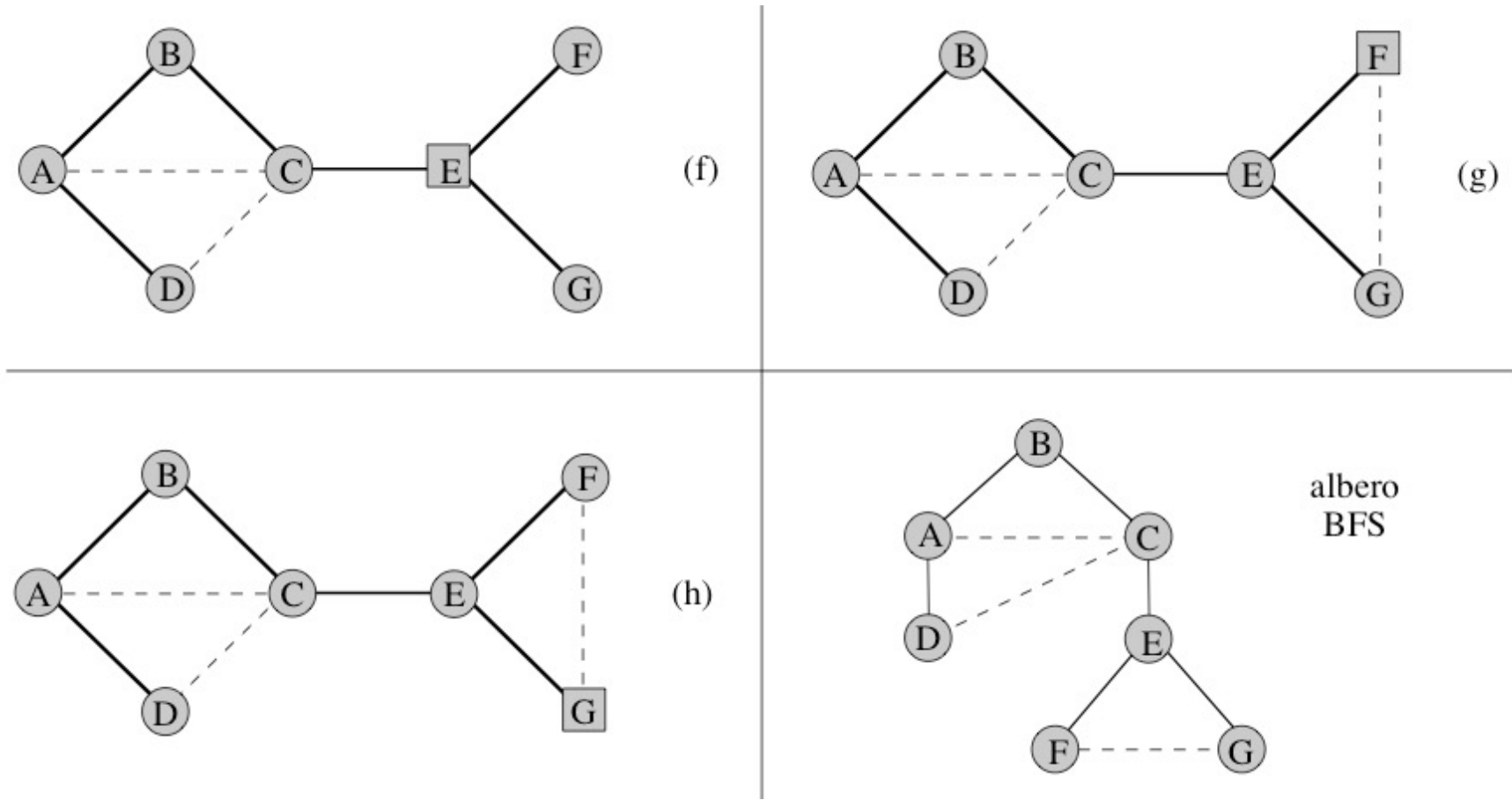
Versione iterativa (con albero di copertura):

```
function iterative_BFS(graph  $G$ , node  $n$ ):  
     $C$  = empty queue  
     $visited$  = empty list  
     $T$  = empty tree  
    enqueue  $n$  in  $C$   
    add  $n$  to  $visited$   
    while  $C$  is not empty:  
        dequeue  $n$  from  $C$   
        for each edge  $(n, v)$  in  $G$ :  
            if  $v$  not in  $visited$ :  
                add  $v$  to  $visited$   
                enqueue  $v$  in  $C$   
                 $u$  is father of  $v$  in  $T$ 
```

Breadth First Search



Breadth First Search



Breath First Search

Proprietà molto utile dell'albero BFS:

- Per ogni nodo v del grafo, il livello di v nell'albero di copertura BFS è pari alla distanza tra v e la sorgente s
- La **distanza** tra s e v è il numero di archi coinvolti nel cammino più breve da s a v
- Vedremo poi come cambia la "distanza" tra nodi nel caso di grafi **pesati** ...

Depth First Search



Depth First Search

Descrizione ad alto livello:

1. si parte dalla radice r
2. si visitano i nodi di figlio in figlio
fino a che non si raggiunge una foglia
3. si torna indietro all'antenato che ha figli inesplorati
 - se non esiste, l'algoritmo termina
4. si ripete da step #3 per ogni figlio

Due possibili implementazioni:

- ricorsiva
- iterativa

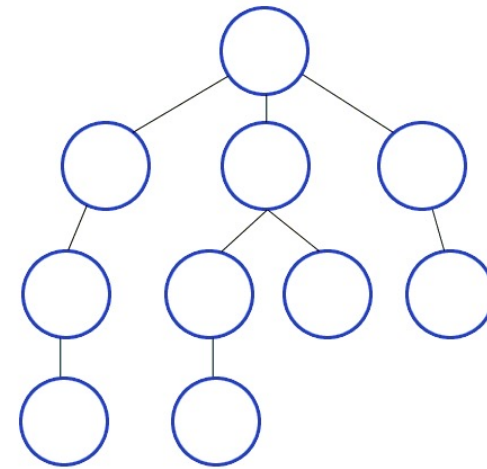


Figura 6: Ricerca in profondità (Depth First Search)

**I LOVE RECURSION,
SAY IT AGAIN**



Depth First Search

Versione ricorsiva per alberi binari:

```
function recursive_DFS(node n):  
    if n is NULL then:  
        return;  
    mark n as visited;  
    recursive_DFS(left child of n);  
    recursive_DFS(right child of n);
```

Recursive DFS (A)

A è visitato

Recursive DFS (L)

L è visitato

Recursive DFS (E)

E è visitato

Recursive DFS (R)

R è visitato

Recursive DFS (B)

B è visitato

non ho figli a sinistra

Recursive DFS (O)

O è visitato

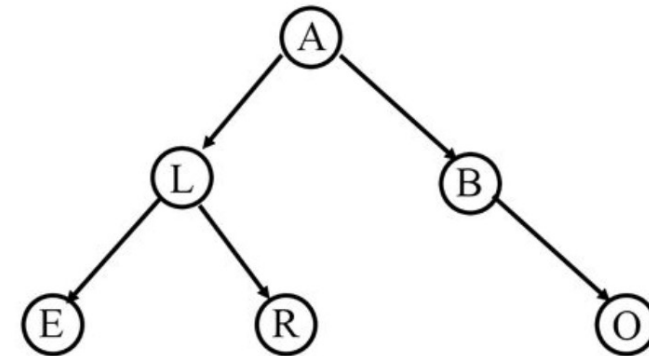


Figura 7: Esempio DFS

Depth First Search

Versione ricorsiva (generalizzata):

```
function recursive_DFS(node n):  
    mark n as visited;  
    for each node v adjacent to n:  
        if v is not visited:  
            recursive_DFS(node v);
```

Recursive DFS (0)
0 è visitato
1, 2 e 3 sono adiacenti a 0
Recursive DFS (1)
1 è visitato
2 e 0 sono adiacenti a 1, ma 0 è visitato
Recursive DFS (2)
2 è visitato
1, 0 e 4 sono adiacenti a 2, ma 1 e 0 sono visitati
Recursive DFS (4)
4 è visitato
2 è adiacente a 4, ma 2 è visitato

Recursive DFS (3)
3 è visitato

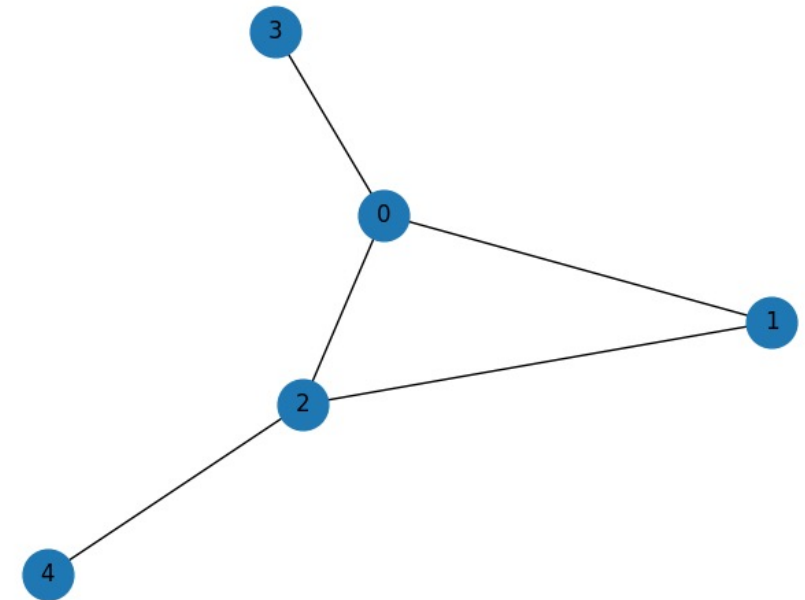


Figura 8: Esempio DFS

Depth First Search

Versione iterativa:

```
function iterative_DFS(graph G, node n):  
    S = empty stack  
    visited = empty list  
    push n to S  
    while S is not empty:  
        pop n from S  
        add n to visited  
        for each node u adjacent to n:  
            push u to S
```

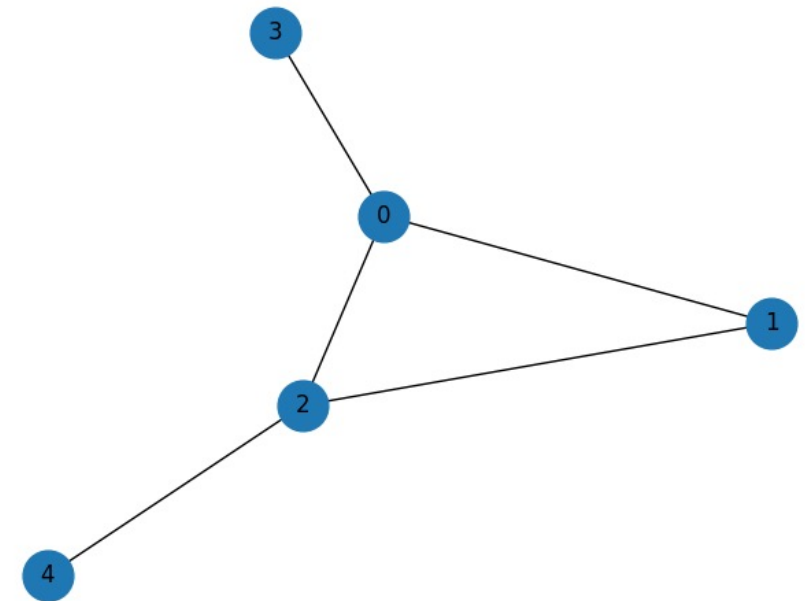


Figura 8: Esempio DFS

Depth First Search

Inizializzazione

stack = [0]

visited = []

Iterazione #1

stack = [1, 2, 3]

visited = [0]

Iterazione #2

stack = [1, 2]

visited = [0, 3]

Iterazione #3

stack = [1, 4]

visited = [0, 3, 2]

Iterazione #4

stack = [1]

visited = [0, 3, 2, 4]

Iterazione #5

stack = []

visited = [0, 3, 2, 4, 1]

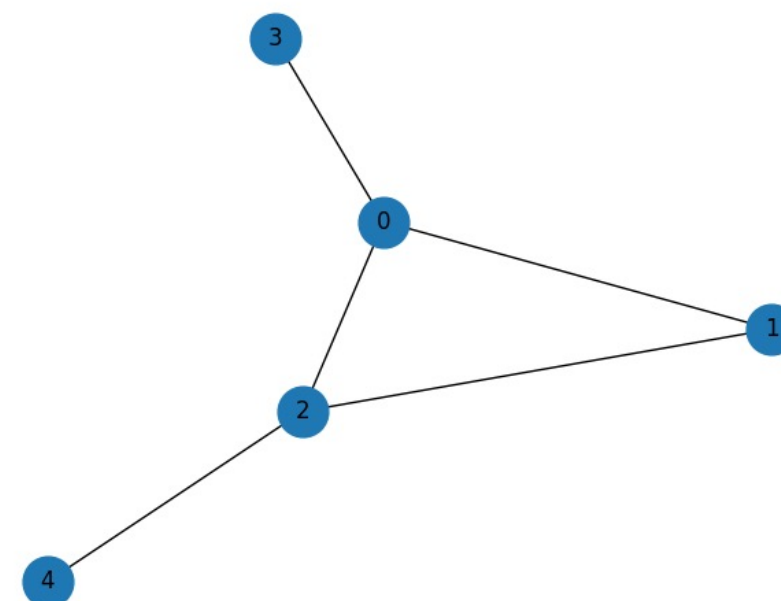


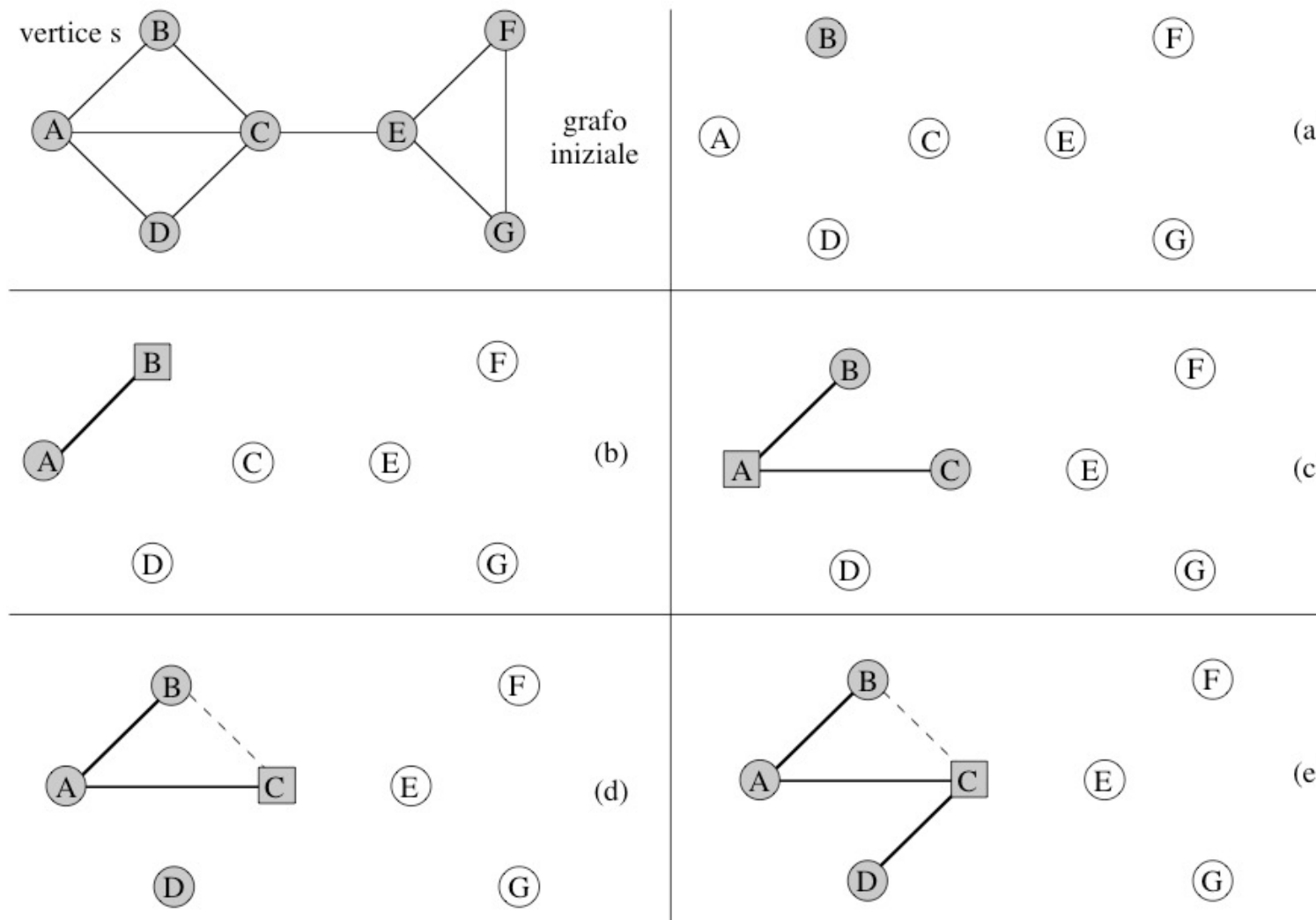
Figura 8: Esempio DFS

Depth First Search

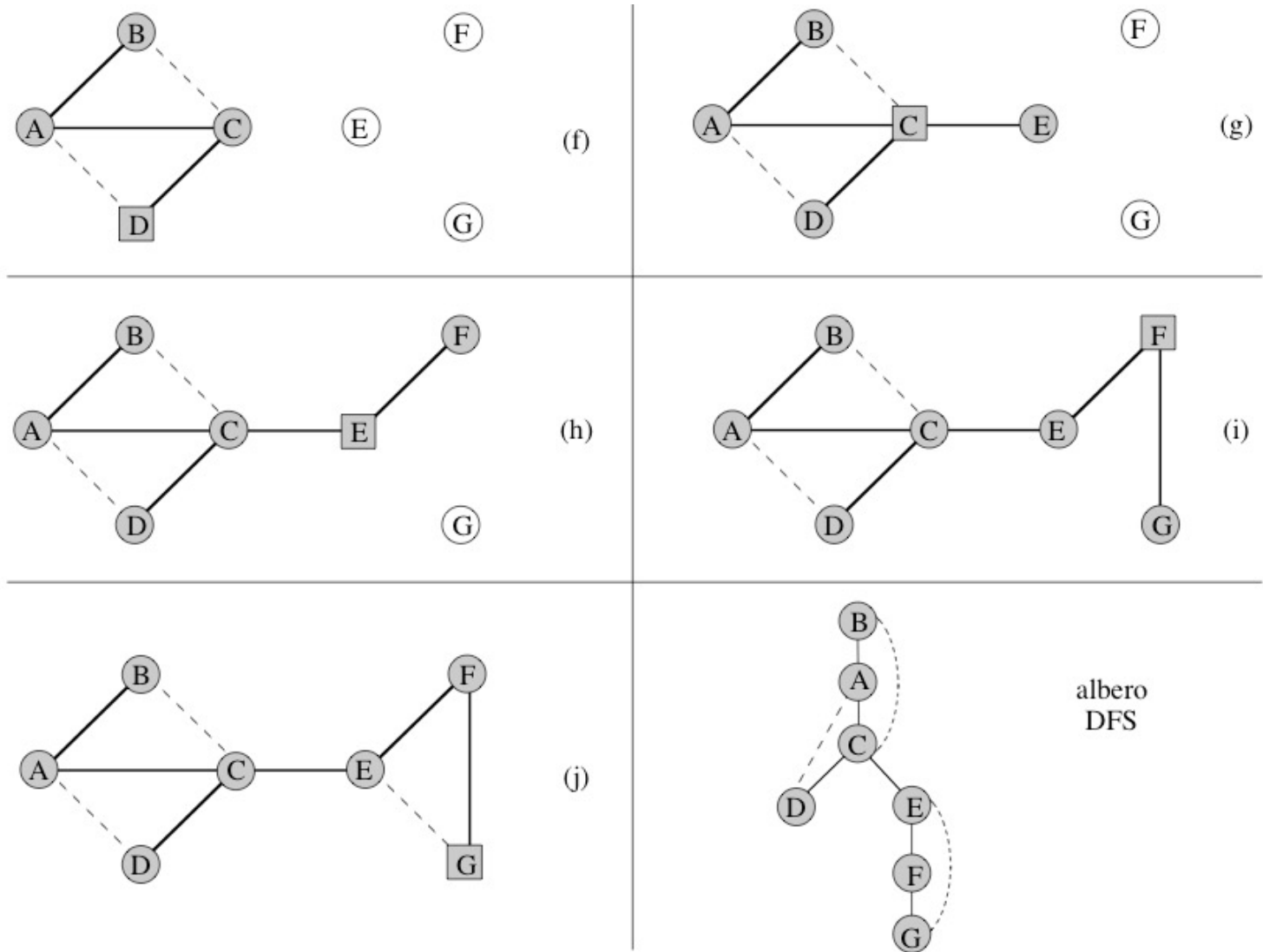
Versione ricorsiva (con albero di copertura):

```
function recursive_DFS(node  $n$ , tree  $T$ , graph  $G$ ):  
    mark  $n$  as visited;  
    for each edge  $(n,v)$  in  $G$ :  
        if  $v$  is not visited:  
            add  $(n,v)$  in  $T$   
            recursive_DFS(node  $v$ , tree  $T$ , grafo  $G$ )  
  
driver DFS(node  $n$ , grafo  $G$ ):  
     $T$  = empty tree  
    recursive_DFS(node  $n$ , tree  $T$ , grafo  $G$ )  
    return  $T$ 
```

Depth First Search



Depth First Search



Problema

Maree a Venezia (OII 2013)

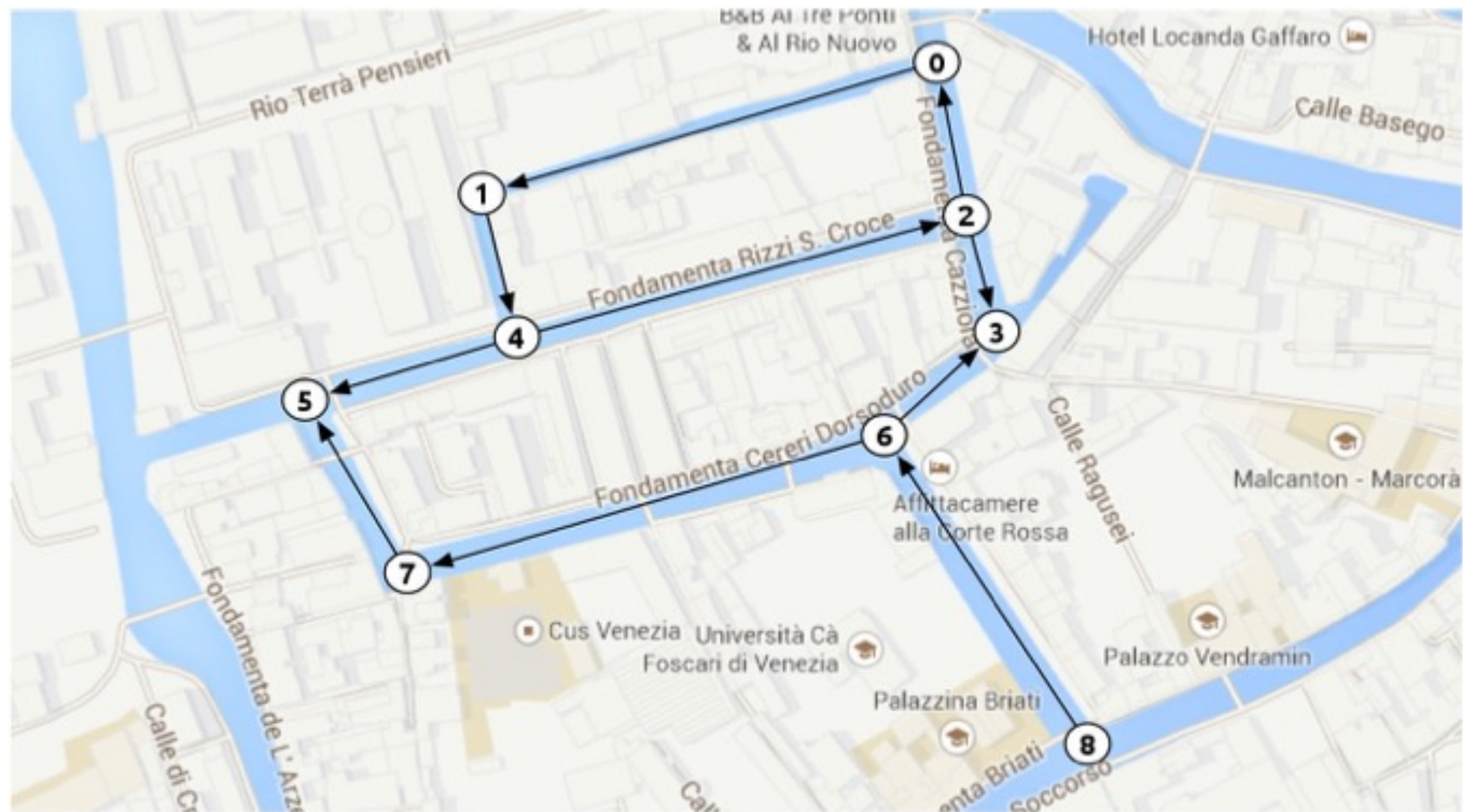
<https://training.olinfo.it/#/task/maree/statement>



I gondolieri di Venezia si spostano lungo i canali, seguendo la corrente. Ogni canale è quindi percorribile in una sola direzione, nota ai gondolieri. Quando però arriva l'alta marea, le dinamiche della laguna sono tali che il verso della corrente di ogni canale si inverte. Dovete aiutare i gondolieri a pianificare il loro percorso per spostarsi in gondola da un punto di origine a un punto di destinazione, sapendo che sta per arrivare l'alta marea.

Data la mappa degli N punti di intersezione e degli M canali della città di Venezia che li collegano, con le correnti che inizialmente ne fluiscono, dovete determinare il minimo tempo necessario per spostarsi in gondola dal punto 0 al punto $N - 1$, tenendo presente che:

- per attraversare un canale – cosa possibile solo nel senso della corrente – si impiega esattamente 1 minuto;
- dopo T minuti arriva l'alta marea e quindi la direzione di percorrenza di tutti i canali si inverte;
- nei punti di intersezione di due o più canali è possibile fermarsi per un numero di minuti a piacere.



Esempio di input/output

File input.txt	File output.txt
9 10 5 0 1 1 4 4 2 2 0 2 3 4 5 7 5 6 7 6 3 8 6	7

Nota: l'esempio riportato è relativo alla situazione mostrata in figura.

Bae: Come over

Dijkstra: But there are so many routes to take and
I don't know which one's the fastest

Bae: My parents aren't home

Dijkstra:

Algoritmo di Dijkstra

Dijkstra's algorithm

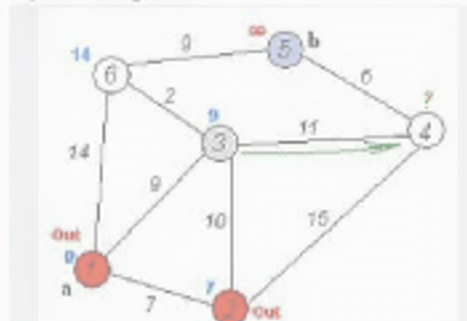
Graph search algorithm

Not to be confused with Dykstra's projection algorithm.

Dijkstra's algorithm is an **algorithm** for finding the **shortest paths** between **nodes** in a **graph**, which may represent, for example, road networks. It was conceived by **computer scientist** **Edsger W. Dijkstra** in 1956 and published three years later.^{[1][2]}

The algorithm exists in many variants; Dijkstra's original variant found the shortest path between two nodes,^[2] but a more common variant fixes a single node as the "source" node and finds shortest paths from the source to all other nodes in the graph, producing a **shortest-path tree**.

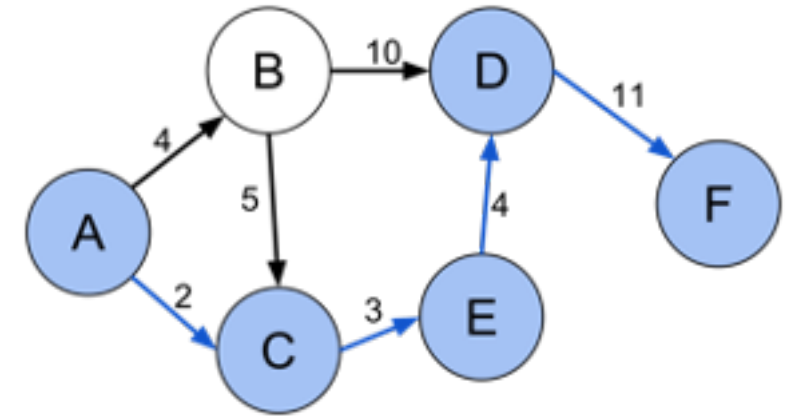
Dijkstra's algorithm



Cammini e costi

Sia $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ un grafo orientato

- $\mathcal{V}$ è l'insieme di vertici
 - $\mathcal{E}$ è l'insieme degli archi
 - ogni arco è formato da una tupla (s, t, w) con
 - s nodo sorgente
 - t nodo destinazione
 - w è un costo (scalare non negativo)
-
- Il **costo di un cammino** è la somma dei costi degli archi coinvolti nel cammino
 - Il **cammino minimo** tra due nodi x ed y è il cammino di costo minore o uguale a quello di ogni altro cammino tra x ed y



Cammini e costi

- La **distanza** $d_{x,y}$ tra due vertici x ed y equivale a:
 - il costo del cammino minimo tra x ed y
 - oppure ∞ se i due nodi non sono connessi

- In matematica, la **disuguaglianza triangolare** ci dice che

$$d_{x,z} \leq d_{x,y} + d_{y,z}$$

- Da cui la **Condizione di Bellman**: per ogni arco (u, v) ed un vertice s

$$d_{s,u} + c(u, v) \geq d_{s,v}$$

dove con $c(u, v)$ indichiamo il costo dell'arco tra u e v

$$d(A, B) \leq d(A, C) + d(C, B)$$

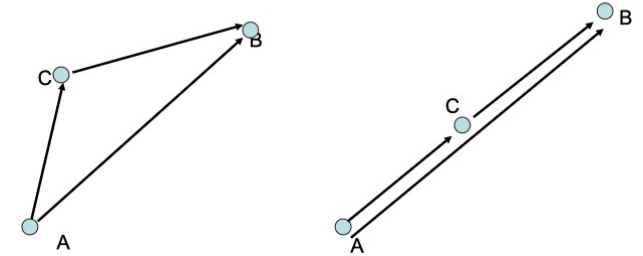


Figura 9: Disuguaglianza Triangolare

Albero dei cammini minimi

Un arco (u, v) appartiene ad un cammino minimo a partire da un vertice s se:

1. u è raggiungibile da s
2. $d_{s,u} + c(u, v) = d_{s,v}$

I cammini minimi da un vertice s a tutti gli altri vertici del grafo possono essere rappresentati tramite un albero radicato in s , detto **albero dei cammini minimi**.

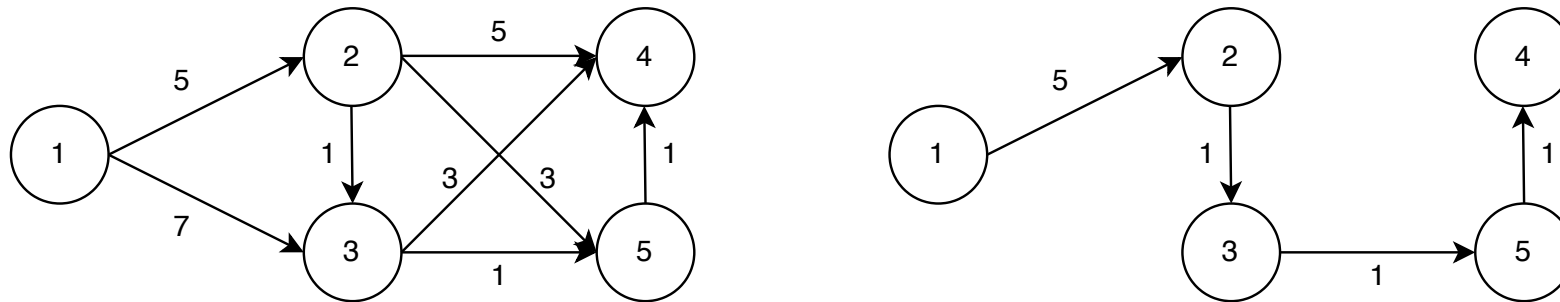


Figura 10: Albero dei cammini minimi radicato in 1

Tecnica del Rilassamento

Partendo da **stime per eccesso** delle distanze $d_{x,y}$ si aggiornano le stime, decrementandole progressivamente fino a renderle **esatte**.

Aggiornamento delle distanze sulla base della seguente regola di rilassamento:

if $d_{x,u} + c(u, v) < d_{x,y}$
then $d_{x,y} = d_{x,u} + c(u, v)$

Questa regola di rilassamento è alla base dell'**approccio di Dijkstra**:

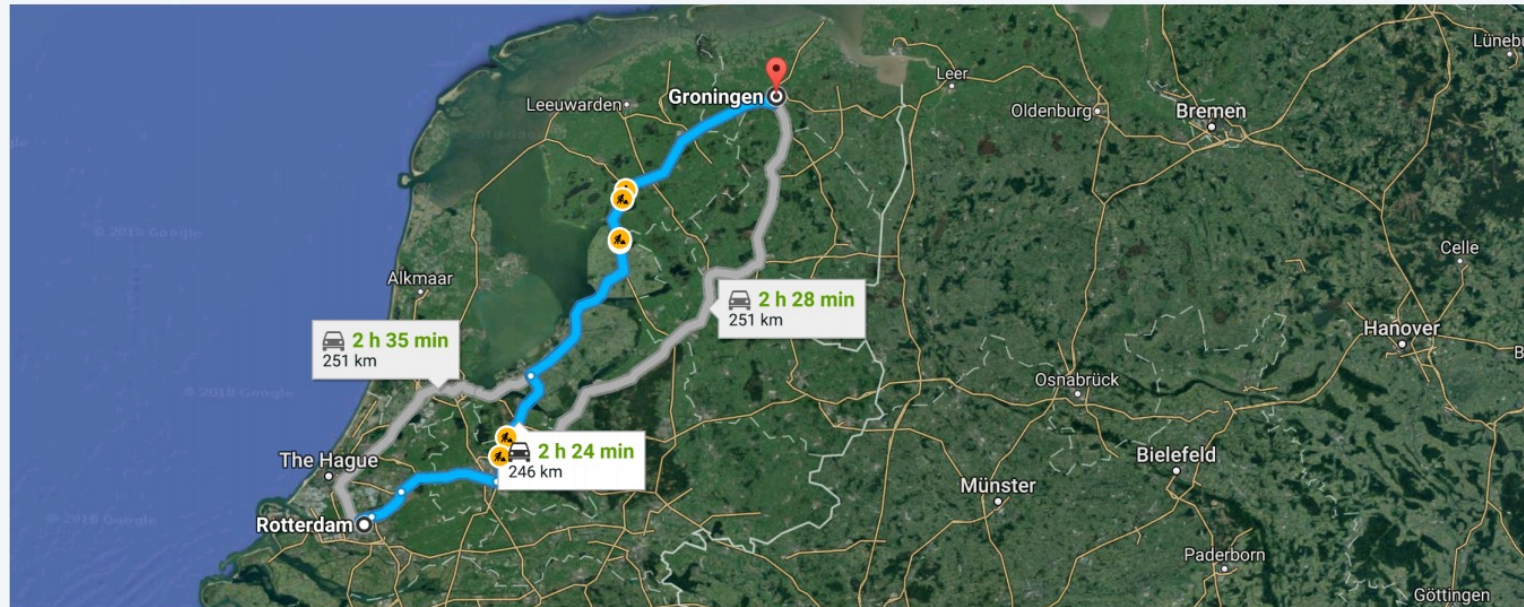
scegli un arco (u, v) con $u \in T$ e $v \notin T$ tale da minimizzare la quantità

$$d_{s,u} + c(u, v)$$

e poi effettua il passo di rilassamento, aggiungendolo a T

$$d_{s,v} = d_{s,u} + c(u, v)$$

*“What’s the shortest way to travel from Rotterdam to Groningen?
It is the algorithm for the shortest path, which I designed in
about 20 minutes. One morning I was shopping in Amsterdam
with my young fiancée, and tired, we sat down on the café
terrace to drink a cup of coffee and I was just thinking about
whether I could do this, and I then designed the algorithm for
the shortest path.” — Edsger Dijkstra*



Algoritmo di Dijkstra (v1.0)

function dijkstra(node s , graph G):

Q = empty list (or set)

for each vertex v in G :

$\text{dist}[v] = \text{INFTY}$

$\text{prev}[v] = \text{NULL}$

add v to Q

$\text{dist}[s] = 0$

$\text{dist}[v]$ contiene la
distanza tra s e v

$\text{prev}[v]$ contiene il
nodo precedente a v

while Q is not empty:

u = node from Q with minimum $\text{dist}[u]$

delete u from Q

for each (u, v) with v in Q :

$\text{temp} = \text{dist}[u] + c(u, v)$

if $\text{temp} < \text{dist}[v]$:

$\text{dist}[v] = \text{temp}$

$\text{prev}[v] = u$

calcolo la distanza tra s ed
 v qualora passassi per u

ho trovato un path più
breve?

Algoritmo di Dijkstra (v1.0)

```
function dijkstra(node  $s$ , graph  $G$ ):
```

```
     $Q$  = empty list (or set)
```

```
    for each vertex  $v$  in  $G$ :
```

```
         $\text{dist}[v]$  = INFTY
```

```
         $\text{prev}[v]$  = NULL
```

```
        add  $v$  to  $Q$ 
```

```
     $\text{dist}[s]$  = 0
```

```
    while  $Q$  is not empty:
```

```
         $u$  = node from  $Q$  with minimum  $\text{dist}[u]$ 
```

```
        delete  $u$  from  $Q$ 
```

```
        for each  $(u, v)$  with  $v$  in  $Q$ :
```

```
             $\text{temp}$  =  $\text{dist}[u]$  +  $c(u, v)$ 
```

```
            if  $\text{temp}$  <  $\text{dist}[v]$ :
```

```
                 $\text{dist}[v]$  =  $\text{temp}$ 
```

```
                 $\text{prev}[v]$  =  $u$ 
```

$$\mathcal{O}(n \cdot n + m) = \mathcal{O}(n^2)$$

n = numero di nodi

m = numero di archi

itero su ciascun
nodo

ricerca lineare per il
minimo tra i nodi

itero su ciascun arco

Algoritmo di Dijkstra (v2.0)

```
function dijkstra(node  $s$ , graph  $G$ ):
```

```
     $Q$  = empty min-priority queue
```

```
    dist[ $s$ ] = 0
```

```
     $Q$ .add( $s$ , dist[ $s$ ])
```

```
    for each vertex  $v \neq s$  in  $G$ :
```

```
        dist[ $v$ ] = INFTY
```

```
        prev[ $v$ ] = NULL
```

```
         $Q$ .add( $v$ , dist[ $v$ ])
```

```
    while  $Q$  is not empty:
```

```
         $u$  =  $Q$ .take_minimum()
```

```
        for each  $(u, v)$  in  $G$ :
```

```
            temp = dist[ $u$ ] +  $c(u, v)$ 
```

```
            if temp < dist[ $v$ ]:
```

```
                dist[ $v$ ] = temp
```

```
                prev[ $v$ ] =  $u$ 
```

```
                 $Q$ .change_priority( $v$ , temp)
```

$$O((\log_2 n) \cdot n + m)$$

n = numero di nodi

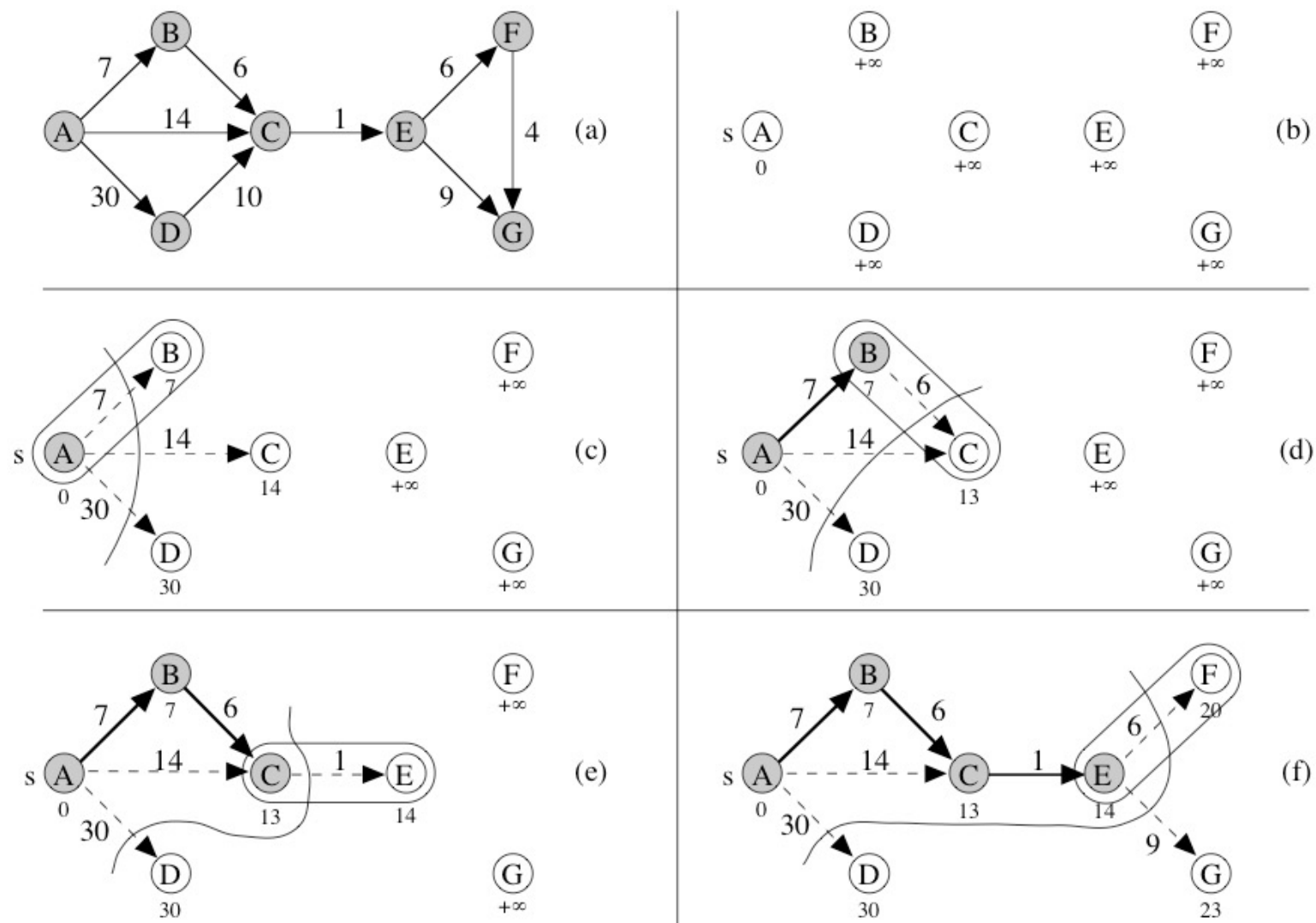
m = numero di archi

itero su ciascun
nodo

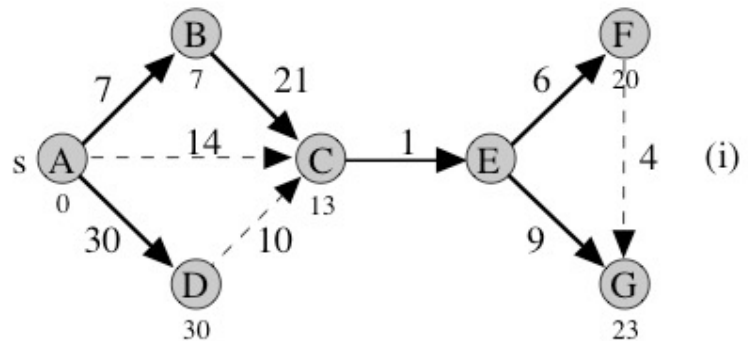
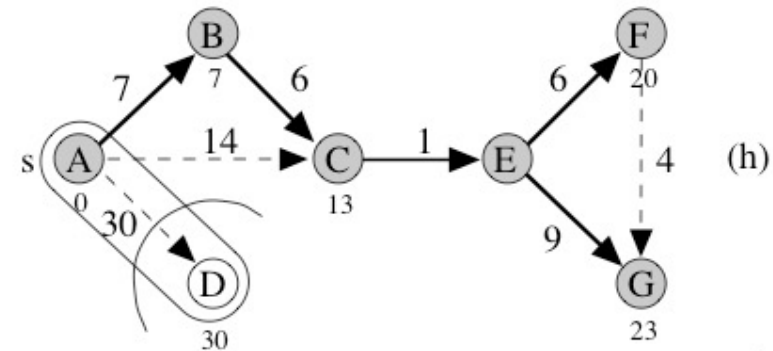
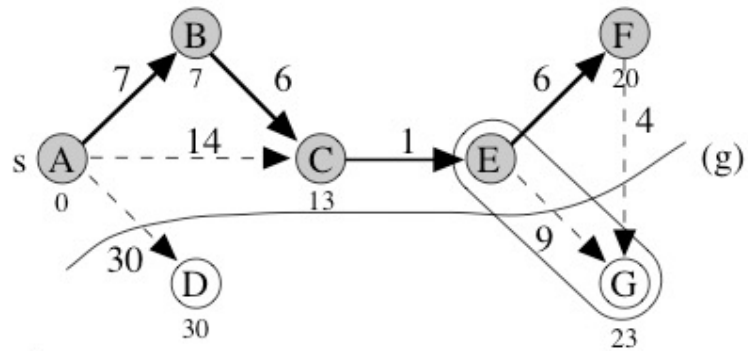
ricerca logaritmica per il
minimo tra i nodi

itero su ciascun arco

Esempio



Esempio



Problema

Corsetta mattutina (Territoriali 2015)

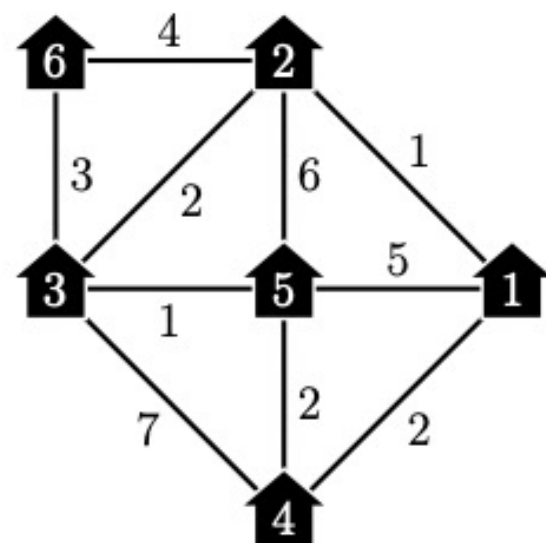
<https://training.olinfo.it/#/task/footing/statement>



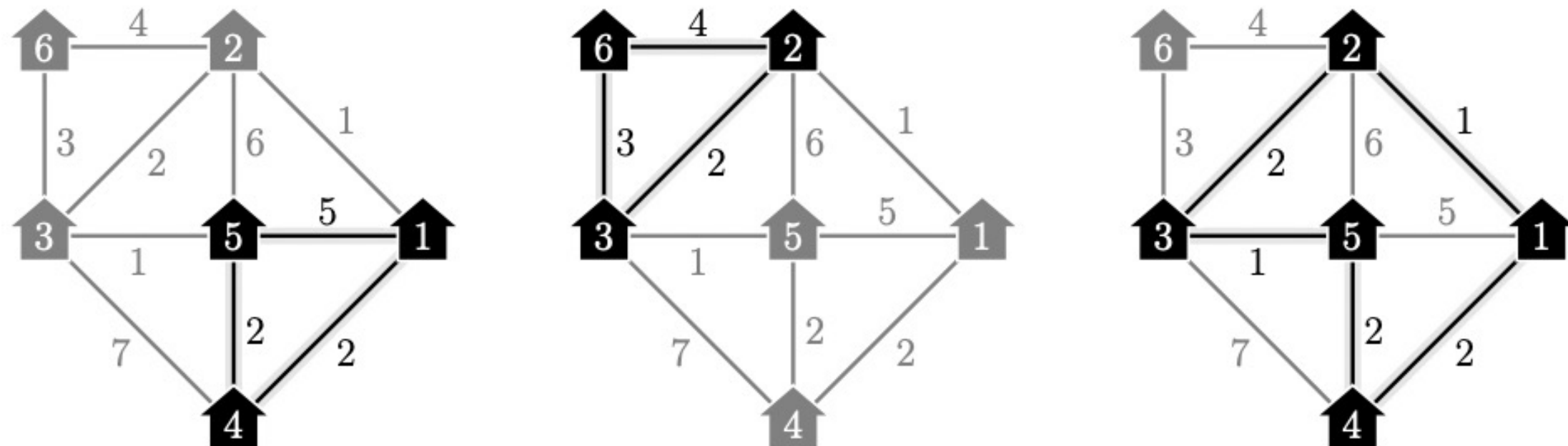
William sta pensando di trasferirsi in una nuova città e vuole selezionare, tra le varie possibilità, quella che si concilia meglio con la sua routine mattutina. Infatti, William è abituato a fare una corsetta attorno al proprio isolato tutte le mattine, e teme che traslocando debba rinunciare a questo hobby, qualora l'isolato in cui verrebbe a trovarsi fosse troppo grande.

La mappa della città si può rappresentare come un insieme di strade e di incroci tra queste. A ogni incrocio c'è una casa e le strade possono essere percorse in entrambi i sensi. Le case sono numerate da 1 a N . Per evitare di annoiarsi, William non ha intenzione di fare corsette che passino due volte davanti alla stessa casa, ad eccezione della sua (infatti la corsetta deve necessariamente cominciare e terminare nella stessa casa). Questo tipo di percorso prende il nome di *ciclo semplice*.

Nonostante i buoni propositi, William è molto pigro; per questo motivo ha intenzione di rendere la sua corsetta mattutina il più breve possibile: aiutalo scrivendo un programma che prenda in input la mappa di una città e determini la lunghezza del *ciclo semplice* più corto. Con questa informazione, William potrà decidere se trasferirsi nella nuova città, ovviamente solo se riuscirà poi ad andare ad abitare in una delle case che appartengono a questo percorso.



Si prenda ad esempio la mappa della città in Figura 1 (dove il numero a fianco di ogni strada indica la lunghezza della strada), alcuni dei suoi cicli semplici sono i seguenti:



Come si può vedere, i primi due cicli evidenziati hanno una lunghezza totale pari a 9, il terzo invece ha una lunghezza pari a 8 ed è quindi il percorso ottimale per la corsetta mattutina di William: adesso William sa quali sono le case coinvolte nel percorso più breve, e tra quelle potrà cercare la nuova casa in cui andare ad abitare.

Assunzioni

- $3 \leq N \leq 1000$.
- $3 \leq M \leq 10\,000$.
- $0 < w \leq 10\,000$, dove w è la lunghezza di un tratto di strada.
- È garantito che nella città esiste sempre almeno un ciclo semplice.
- Nel 40% dei casi di prova tutte le strade hanno lunghezza unitaria.
- È garantito che una coppia di case adiacenti è collegata da *un solo* tratto di strada.
- Una strada non collega mai una casa a se stessa.

Esempi di input/output

input.txt	output.txt
6 10 1 2 1 3 2 2 5 2 6 4 5 2 1 4 2 3 5 1 3 4 7 5 1 5 2 6 4 3 6 3	8

Esercizi per casa

1. Sentieri bollenti

<https://training.olinfo.it/#/task/sentieri/statement>

2. Giochiamo con Mojito

<https://training.olinfo.it/#/task/mojito/statement>

3. Il tesoro del Pirata Barbablù

<https://training.olinfo.it/#/task/barbablu/statement>

4. Tedoforo

<https://training.olinfo.it/#/task/tedoforo/statement>

5. Fulcro del gioco

<https://training.olinfo.it/#/task/fulcrodelgioco/statement>